UML Class Diagram – Insurance Claims System (ICS)

Packages: ics · ics.model · ics.repository · ics.service · ics.exception · ics.util · ics.gui | + public - private # protected ~ package | attribute: visibility name : type = default | method: visibility name(parameters) : return_type | underline = static | italic = abstract | Outer.Inner = nested class | 62 classes

«abstract»
Entity
ics.model

```
- id : String

# Entity(id: String)
+ getId() : String
+ setId(id: String) : void
+ toFileLine() : String
```

«abstract»
User
ics.model · extends Entity

```
+ MAX_FAILED_ATTEMPTS : int = 3
- username : String
- passwordHash : String
- fullName : String
- contact : String
- address : String
- failedAttempts : int
- blocked : boolean

# User(id: String, username: String, passwordHash: String, fullName: String, contact: String, address: String, failedAttempts: int, blocked: boolean)
+ block() : void
+ canApplyForClaims() : boolean
+ canManageUsers() : boolean
+ canReviewClaims() : boolean
+ create(role: User.Role, id: String, username: String, passwordHash: String, fullName: String, contact: String, address: String, failedAttempts: int, blocked: boolean) : User
+ fromFields(f: String[]) : User
+ getAddress() : String
+ getContact() : String
+ getFailedAttempts() : int
+ getFullName() : String
+ getPasswordHash() : String
+ getRole() : User.Role
+ getUsername() : String
+ isBlocked() : boolean
+ recordFailedAttempt() : void
+ resetFailedAttempts() : void
+ setAddress(address: String) : void
+ setContact(contact: String) : void
+ setFullName(fullName: String) : void
+ setPasswordHash(passwordHash: String) : void
+ setUsername(username: String) : void
+ toFileLine() : String
+ unblock() : void
+ verifyPassword(typedHash: String) : boolean
```

Admin
ics.model · extends User

```
+ Admin(id: String, username: String, passwordHash: String, fullName: String, contact: String, address: String, failedAttempts: int, blocked: boolean)
+ canApplyForClaims() : boolean
+ canManageUsers() : boolean
+ canReviewClaims() : boolean
+ getRole() : User.Role
```

Applicant
ics.model · extends User

```
+ Applicant(id: String, username: String, passwordHash: String, fullName: String, contact: String, address: String, failedAttempts: int, blocked: boolean)
+ canApplyForClaims() : boolean
+ canManageUsers() : boolean
+ canReviewClaims() : boolean
+ getRole() : User.Role
```

«enumeration»
User.Role
ics.model

```
ADMIN
USER
```

«abstract»
Policy
ics.model · extends Entity

```
+ DISCOUNT_DEDUCTIBLE : double = 10_000
+ DISCOUNT_RATE : double = 0.10
- ownerId : String
- startDate : LocalDate
- endDate : LocalDate
- deductible : double
- coverageLimit : double
- limitUsed : double
- cancelled : boolean

# Policy(id: String, ownerId: String, startDate: LocalDate, endDate: LocalDate, deductible: double, coverageLimit: double, limitUsed: double, cancelled: boolean)
+ applyPayout(claimedAmount: double, paidAmount: double) : void
+ calculatePayout(claimedAmount: double) : double
+ calculatePremium() : double
+ cancel() : void
+ explainPayout(claimedAmount: double) : List<String>
+ fromFields(f: String[]) : Policy
+ getCoverageLimit() : double
+ getDeductible() : double
+ getEndDate() : LocalDate
+ getLimitUsed() : double
+ getOwnerId() : String
+ getRemainingLimit() : double
+ getStartDate() : LocalDate
+ getStatus(today: LocalDate) : Policy.Status
+ getType() : Policy.Type
+ hasDeductibleDiscount() : boolean
+ isCancelled() : boolean
+ payoutFormula(claimedAmount: double, currencySign: String) : String
+ toFileLine() : String
# amount(currencySign: String, value: double) : String
# extraFields() : Object[]
# php(value: double) : String
# roundToCents(value: double) : double
- setPeriod(startDate: LocalDate, endDate: LocalDate) : void
# withDeductibleDiscount(basePremium: double) : double
```

HealthPolicy
ics.model · extends Policy

```
+ PREMIUM_RATE : double = 0.04
- reimbursementRate : double
- waitingPeriodDays : int
- deductibleMet : double

+ HealthPolicy(id: String, ownerId: String, startDate: LocalDate, endDate: LocalDate, deductible: double, coverageLimit: double, limitUsed: double, cancelled: boolean, reimbursementRate: double, waitingPeriodDays: int, deductibleMet: double)
+ applyPayout(claimedAmount: double, paidAmount: double) : void
+ calculatePayout(claimedAmount: double) : double
+ calculatePremium() : double
+ explainPayout(claimedAmount: double) : List<String>
+ getDeductibleMet() : double
+ getReimbursementRate() : double
+ getRemainingDeductible() : double
+ getType() : Policy.Type
+ getWaitingPeriodDays() : int
+ getWaitingPeriodEnd() : LocalDate
+ isInWaitingPeriod(incidentDate: LocalDate) : boolean
+ payoutFormula(claimedAmount: double, c: String) : String
# extraFields() : Object[]
```

AutoPolicy
ics.model · extends Policy

```
+ PREMIUM_RATE : double = 0.03
- vehicleValue : double
- plateNo : String

+ AutoPolicy(id: String, ownerId: String, startDate: LocalDate, endDate: LocalDate, deductible: double, coverageLimit: double, limitUsed: double, cancelled: boolean, vehicleValue: double, plateNo: String)
+ calculatePayout(claimedAmount: double) : double
+ calculatePremium() : double
+ explainPayout(claimedAmount: double) : List<String>
+ getPlateNo() : String
+ getType() : Policy.Type
+ getVehicleValue() : double
+ payoutFormula(claimedAmount: double, c: String) : String
# extraFields() : Object[]
```

«enumeration»
Policy.Type
ics.model

```
HEALTH("Health")
AUTO("Auto")
- label : String

+ getLabel() : String
```

«enumeration»
Policy.Status
ics.model

```
UPCOMING("Upcoming")
ACTIVE("Active")
EXPIRED("Expired")
CANCELLED("Cancelled")
- label : String

+ getLabel() : String
```

Claim
ics.model · extends Entity

```
+ MAX_NOTE_LENGTH : int = 300
- policyId : String
- incidentDate : LocalDate
- filedDate : LocalDate
- description : String
- claimedAmount : double
- reserveAmount : double
- approvedAmount : double
- paidAmount : double
- status : Claim.Status
- reason : Claim.Reason
- handledBy : String
- reviewerNote : String
- applicantNote : String

+ Claim(id: String, policyId: String, incidentDate: LocalDate, filedDate: LocalDate, description: String, claimedAmount: double, reserveAmount: double, approvedAmount: double, paidAmount: double, status: Claim.Status, reason: Claim.Reason, handledBy: String, reviewerNote: String, applicantNote: String)
+ Claim(policyId: String, incidentDate: LocalDate, filedDate: LocalDate, description: String, claimedAmount: double)
+ approve(amount: double, adminId: String) : void
+ changeStatus(next: Claim.Status) : void
+ deny(why: Claim.Reason, adminId: String) : void
+ fromFields(f: String[]) : Claim
+ getApplicantNote() : String
+ getApprovedAmount() : double
+ getClaimedAmount() : double
+ getDescription() : String
+ getFiledDate() : LocalDate
+ getHandledBy() : String
+ getIncidentDate() : LocalDate
+ getPaidAmount() : double
+ getPolicyId() : String
+ getReason() : Claim.Reason
+ getReserveAmount() : double
+ getReviewerNote() : String
+ getStatus() : Claim.Status
+ reject(why: Claim.Reason) : void
+ setApplicantNote(note: String) : void
+ setHandledBy(adminId: String) : void
+ setReserveAmount(amount: double) : void
+ setReviewerNote(note: String) : void
+ toFileLine() : String
- checkNote(field: String, note: String) : String
```

«enumeration»
Claim.Status
ics.model

```
FILED("Filed")
REJECTED("Rejected")
UNDER_REVIEW("Under review")
INFO_REQUESTED("Info requested")
APPROVED("Approved")
DENIED("Denied")
PAID("Paid")
CLOSED("Closed")
- label : String

+ canMoveTo(next: Claim.Status) : boolean
+ getLabel() : String
```

«enumeration»
Claim.Reason
ics.model

```
POLICY_INACTIVE("Policy is not active")
OUTSIDE_COVERAGE_PERIOD("Incident is outside the coverage period")
WAITING_PERIOD("Incident happened inside the waiting period")
LATE_FILING("Claim was filed too late")
DUPLICATE_CLAIM("Same incident was already claimed")
LIMIT_EXHAUSTED("Coverage limit is used up")
EXCLUDED_CAUSE("Cause of loss is not covered")
SUSPECTED_FRAUD("Claim failed the fraud review")
INSUFFICIENT_DOCUMENTS("Supporting documents are missing")
- label : String

+ getLabel() : String
```

Payout
ics.model · extends Entity

```
- claimId : String
- amount : double
- paidDate : LocalDate
- paidBy : String
- breakdown : String

+ Payout(id: String, claimId: String, amount: double, paidDate: LocalDate, paidBy: String, breakdown: String)
+ fromFields(f: String[]) : Payout
+ getAmount() : double
+ getBreakdown() : String
+ getClaimId() : String
+ getPaidBy() : String
+ getPaidDate() : LocalDate
+ toFileLine() : String
```

Repository
ics.repository

```
- items : ArrayList<T> = new ArrayList<>()
- file : Path
- prefix : String
- reader : Repository.LineReader<T>
- warnings : List<String> = new ArrayList<>()

+ Repository(file: Path, prefix: String, reader: Repository.LineReader<T>)
+ add(item: T) : void
+ delete(id: String) : void
+ findById(id: String) : T
+ getAll() : List<T>
+ getWarnings() : List<String>
+ load() : void
+ nextId() : String
+ save() : void
+ update(item: T) : void
- indexOf(id: String) : int
- putBack(before: ArrayList<T>) : void
```

«interface»
Repository.LineReader
ics.repository

```
+ read(fields: String[]) : T
```

DataStore
ics.repository

```
- users : Repository<User>
- policies : Repository<Policy>
- claims : Repository<Claim>
- payouts : Repository<Payout>

+ DataStore(folder: Path)
+ claims() : Repository<Claim>
+ claimsOn(policyId: String) : List<Claim>
+ findUserByUsername(username: String) : User
+ loadAll() : void
+ loadWarnings() : List<String>
+ payoutFor(claimId: String) : Payout
+ payouts() : Repository<Payout>
+ policies() : Repository<Policy>
+ policiesOf(ownerId: String) : List<Policy>
+ users() : Repository<User>
```

AuthService
ics.service

```
+ DEFAULT_ADMIN_USERNAME : String = "admin"
+ DEFAULT_ADMIN_PASSWORD : String = "Admin123"
- store : DataStore

+ AuthService(store: DataStore)
+ changePassword(user: User, currentPassword: String, newPassword: String, confirmPassword: String) : void
+ confirmPassword(admin: User, typedPassword: String) : void
+ ensureFirstAdmin() : boolean
+ hash(plainText: String) : String
+ isUsingDefaultPassword(user: User) : boolean
+ login(username: String, password: String) : User
```

UserService
ics.service

```
- store : DataStore
- auth : AuthService

+ UserService(store: DataStore)
+ countUsers() : int
+ createAccount(role: User.Role, username: String, password: String, confirmPassword: String, fullName: String, contact: String, address: String) : User
+ deactivate(id: String, admin: User) : void
+ delete(id: String, admin: User, adminPassword: String) : void
+ search(keyword: String) : List<User>
+ unblock(id: String) : void
+ updateProfile(id: String, fullName: String, contact: String, address: String) : void
- checkProfile(fullName: String, contact: String, address: String) : void
- find(id: String) : User
- protectAdmins(target: User, admin: User, action: String) : void
- requireAdmin(admin: User) : void
```

PolicyService
ics.service

```
- store : DataStore
- auth : AuthService

+ PolicyService(store: DataStore, auth: AuthService)
+ cancel(id: String, admin: User, adminPassword: String) : void
+ countPolicies() : int
+ delete(id: String, admin: User, adminPassword: String) : void
+ holderName(ownerId: String) : String
+ issue(type: Policy.Type, ownerId: String, startDate: String, endDate: String, deductible: String, coverageLimit: String, reimbursementPercent: String, waitingDays: String, vehicleValue: String, plateNo: String) : Policy
+ policyHolders() : List<User>
+ previewPremium(type: Policy.Type, coverageLimit: String, vehicleValue: String, deductibleText: String) : double
+ search(keyword: String) : List<Policy>
+ update(id: String, startDate: String, endDate: String, deductible: String, coverageLimit: String, reimbursementPercent: String, waitingDays: String, vehicleValue: String, plateNo: String) : Policy
- build(id: String, type: Policy.Type, ownerId: String, startDate: String, endDate: String, deductibleText: String, limitText: String, percentText: String, waitingText: String, vehicleText: String, plateText: String, limitUsed: double, deductibleMet: double) : Policy
- checkOverlap(id: String, type: Policy.Type, owner: User, plate: String, start: LocalDate, end: LocalDate) : void
- find(id: String) : Policy
- money(value: double) : String
```

ClaimService
ics.service

```
+ FILING_WINDOW_DAYS : int = 90
- MAX_DESCRIPTION_LENGTH : int = 300
- store : DataStore
- auth : AuthService
- clock : Clock

+ ClaimService(store: DataStore, auth: AuthService)
+ ClaimService(store: DataStore, auth: AuthService, clock: Clock)
+ approve(claimId: String, admin: User, note: String, adminPassword: String) : Payout
+ claimsOf(applicant: User) : List<Claim>
+ claimsOnPolicy(applicant: User, policyId: String) : List<Claim>
+ close(claimId: String, admin: User) : void
+ countAwaitingReply(user: User) : int
+ countClaims() : int
+ countNeedingAction() : int
+ countPayouts() : int
+ deny(claimId: String, admin: User, reason: Claim.Reason, note: String) : void
+ estimate(applicant: User, policyId: String, claimedAmount: String) : double
+ file(applicant: User, policyId: String, incidentDate: String, description: String, claimedAmount: String) : Claim
+ getClaim(claimId: String) : Claim
+ holderName(claim: Claim) : String
+ holderOf(claim: Claim) : User
+ payoutOf(claim: Claim) : Payout
+ payoutSlip(claimId: String) : List<String>
+ payoutSlipFor(applicant: User, claimId: String) : List<String>
+ policiesOf(applicant: User) : List<Policy>
+ policyOf(claim: Claim) : Policy
+ precheck(applicant: User, policyId: String, incidentDate: String) : Claim.Reason
+ requestInfo(claimId: String, admin: User, note: String) : void
+ respond(applicant: User, claimId: String, reply: String) : void
+ resumeReview(claimId: String, admin: User) : void
+ search(keyword: String, filter: ClaimService.Filter) : List<Claim>
+ today() : LocalDate
+ userName(userId: String) : String
- adminGroup(s: Claim.Status) : int
- center(text: String) : String
- cleanNote(note: String) : String
- compareForAdmin(a: Claim, b: Claim) : int
- compareForUser(a: Claim, b: Claim) : int
- compareNewestFirst(a: Claim, b: Claim) : int
- firstBrokenRule(policy: Policy, incident: LocalDate) : Claim.Reason
- money(value: double) : String
- ownClaim(applicant: User, claimId: String) : Claim
- ownPolicy(applicant: User, policyId: String) : Policy
- requireReviewer(admin: User) : void
- requireStarted(policy: Policy) : void
- row(label: String, value: String) : String
- userGroup(s: Claim.Status) : int
```

«enumeration»
ClaimService.Filter
ics.service

```
NEEDS_ACTION("Needs action")
ALL("All claims")
UNDER_REVIEW("Under review")
INFO_REQUESTED("Info requested")
APPROVED("Approved")
PAID("Paid (older claims)")
DENIED("Denied")
REJECTED("Rejected")
CLOSED("Closed")
- label : String

+ accepts(status: Claim.Status) : boolean
+ toString() : String
```

ICSException
ics.exception · extends Exception

```
+ ICSException(message: String)
+ ICSException(message: String, cause: Throwable)
```

InvalidInputException
ics.exception · extends ICSException

```
- fieldName : String

+ InvalidInputException(fieldName: String, message: String)
+ getFieldName() : String
```

RuleException
ics.exception · extends ICSException

```
+ RuleException(message: String)
```

AccessDeniedException
ics.exception · extends ICSException

```
+ AccessDeniedException(message: String)
```

LoginException
ics.exception · extends ICSException

```
- triesLeft : int
- blocked : boolean
- justBlocked : boolean

- LoginException(message: String, triesLeft: int, blocked: boolean, justBlocked: boolean)
+ blocked(justNow: boolean) : LoginException
+ getTriesLeft() : int
+ isBlocked() : boolean
+ isJustBlocked() : boolean
+ wrongPassword(triesLeft: int) : LoginException
```

DataStorageException
ics.exception · extends ICSException

```
+ DataStorageException(message: String)
+ DataStorageException(message: String, cause: Throwable)
```

Validate
ics.util

```
- Validate()
+ between(field: String, value: double, min: double, max: double) : double
+ nonNegative(field: String, value: double) : double
+ notBlank(field: String, value: String) : String
+ notNull(field: String, value: T) : T
+ parseAmount(field: String, text: String) : double
+ parseDate(field: String, text: String) : LocalDate
+ parseWholeNumber(field: String, text: String) : int
+ plainText(field: String, value: String, maxLength: int) : String
+ positive(field: String, value: double) : double
+ strongPassword(field: String, value: String) : String
+ username(field: String, value: String) : String
- plain(value: double) : String
```

FileHelper
ics.util

```
+ FOLDER_NAME : String = "ICS-data"
- FILE_NAMES : String[] = {"users.txt", "policies.txt", "claims.txt", "payouts.txt"}

- FileHelper()
+ dataFolder() : Path
+ join(fields: Object...) : String
+ moveOldDataIfNeeded(newFolder: Path) : boolean
+ split(line: String) : String[]
- unescape(s: String) : String
```

Main
ics · extends JFrame

```
- store : DataStore
- auth : AuthService

+ Main(store: DataStore, auth: AuthService)
+ main(args: String[]) : void
+ showLogin() : void
+ showShell(user: User) : void
```

Theme
ics.gui

```
+ NAVY : Color = new Color(0x4B1C9C)
+ ROSE : Color = new Color(0xA31D72)
+ WHITE : Color = Color.WHITE
+ BACKGROUND : Color = new Color(0xF7F5F7)
+ TEXT_DARK : Color = new Color(0x2B2B2B)
+ TEXT_MUTED : Color = new Color(0x6B6B6B)
+ BORDER : Color = new Color(0xDDD6E5)
+ SOFT_PURPLE : Color = new Color(0xEFE8FA)
+ ERROR : Color = new Color(0xC0392B)
+ SUCCESS : Color = new Color(0x2E8B4F)
+ AMBER : Color = new Color(0x9A6700)
+ AMBER_FILL : Color = new Color(0xFCE9C9)
+ ORANGE : Color = new Color(0xD9650B)
+ GREEN_FILL : Color = new Color(0xE3F2E1)
+ MIN_WINDOW : Dimension = new Dimension(1024, 768)
+ DEFAULT_WINDOW : Dimension = new Dimension(1280, 800)
+ SIDEBAR_WIDTH : int = 240
+ FIELD_SIZE : Dimension = new Dimension(260, 32)
+ BUTTON_SIZE : Dimension = new Dimension(120, 36)
- INSTALLED : List<String> = Arrays.asList(GraphicsEnvironment.getLocalGraphicsEnvironment().getAvailableFontFamilyN…
- SANS : String = pick(Font.SANS_SERIF, "Montserrat", "Segoe UI", "Arial")
- LIGHT : String = pick(SANS, "Montserrat Light")
- MONO : String = pick(Font.MONOSPACED, "JetBrains Mono", "Consolas", "Menlo")
+ FONT_TITLE : Font = new Font(SANS, Font.BOLD, 30)
+ FONT_HEADING : Font = new Font(SANS, Font.BOLD, 22)
+ FONT_STAT : Font = new Font(SANS, Font.BOLD, 34)
+ FONT_LABEL : Font = new Font(SANS, Font.BOLD, 12)
+ FONT_BUTTON : Font = new Font(SANS, Font.BOLD, 13)
+ FONT_BODY : Font = new Font(LIGHT, Font.PLAIN, 14)
+ FONT_CAPTION : Font = new Font(LIGHT, Font.PLAIN, 12)
+ FONT_MONO : Font = new Font(MONO, Font.PLAIN, 11)
- CURRENCY : String = FONT_BODY.canDisplay('\u20B1') ? "\u20B1" : "PHP "

- Theme()
+ card(padding: int) : JPanel
+ currencySign() : String
+ escape(text: String) : String
+ fieldLabel(text: String) : JLabel
+ install() : void
+ label(text: String, font: Font, color: Color) : JLabel
+ money(amount: double) : String
+ requiredLabel(text: String) : JLabel
+ styleCombo(box: JComboBox<?>) : void
+ styleField(field: T) : T
+ styleTable(table: JTable) : void
+ tableRow(html: StringBuilder, label: String, value: String) : void
+ tip(component: T, text: String) : T
+ wrap(text: String, screenPixels: int) : String
+ wrapHtml(html: String, screenPixels: int) : String
- pick(fallback: String, wanted: String...) : String
```

Widgets
ics.gui

```
- Widgets()
+ chip() : JLabel
+ colorFor(status: Claim.Status) : Color
+ colorForPolicy(statusLabel: String) : Color
+ onTextChange(field: JTextComponent, action: Runnable) : void
+ scroll(table: JTable) : JScrollPane
+ scrollDown(content: JComponent) : JScrollPane
+ showWords(box: JComboBox<T>, words: Widgets.WordFor<T>) : void
```

Widgets.Table
ics.gui · extends JTable

```
- model : DefaultTableModel
- refreshing : boolean

+ Table(columns: String[], widths: int[])
+ clearSelectionQuietly() : void
+ colourColumn(column: int, rule: Widgets.ColourRule) : void
+ getScrollableTracksViewportWidth() : boolean
+ onRowSelected(action: Runnable) : void
+ selectQuietly(id: String) : void
+ setRows(rows: List<Object[]>, keepId: String) : void
```

«interface»
Widgets.ColourRule
ics.gui

```
+ colourFor(row: int, text: String) : Color
```

«interface»
Widgets.WordFor
ics.gui

```
+ wordFor(value: T) : String
```

Widgets.RoundedButton
ics.gui · extends JButton

```
- kind : Widgets.RoundedButton.Kind

+ RoundedButton(text: String, kind: Widgets.RoundedButton.Kind)
+ getForeground() : Color
# paintComponent(g: Graphics) : void
```

«enumeration»
Widgets.RoundedButton.Kind
ics.gui

```
PRIMARY
SECONDARY
```

Widgets.NavButton
ics.gui · extends JButton

```
- key : String
- selected : boolean

+ NavButton(key: String)
+ getForeground() : Color
+ getKey() : String
+ setSelected(selected: boolean) : void
# paintComponent(g: Graphics) : void
```

Widgets.ShieldIcon
ics.gui · implements Icon

```
- size : int
- shieldColor : Color
- checkColor : Color

+ ShieldIcon(size: int, shieldColor: Color, checkColor: Color)
+ getIconHeight() : int
+ getIconWidth() : int
+ paintIcon(c: Component, g: Graphics, x: int, y: int) : void
+ toImage() : BufferedImage
```

Widgets.UsageBar
ics.gui · extends JComponent

```
- used : double
- total : double = 1

+ UsageBar()
+ setValues(used: double, total: double) : void
# paintComponent(g: Graphics) : void
```

Widgets.ClaimProgress
ics.gui · extends JComponent

```
- DONE : int = 0
- CURRENT : int = 1
- WAITING : int = 2
- FAILED : int = 3
- PENDING : int = 4
- labels : String[] = new String[4]
- steps : int[] = new int[4]

+ ClaimProgress()
+ showClaim(claim: Claim) : void
- dot(g2: Graphics2D, cx: int, cy: int, r: int, color: Color) : void
# paintComponent(g: Graphics) : void
- set(a: int, b: int, c: int, d: int) : void
```

Widgets.FitWidthPanel
ics.gui · extends JPanel · implements Scrollable

```
+ FitWidthPanel(layout: LayoutManager)
+ getPreferredScrollableViewportSize() : Dimension
+ getScrollableBlockIncrement(r: Rectangle, orientation: int, direction: int) : int
+ getScrollableTracksViewportHeight() : boolean
+ getScrollableTracksViewportWidth() : boolean
+ getScrollableUnitIncrement(r: Rectangle, orientation: int, direction: int) : int
```

Dialogs
ics.gui

```
- Dialogs()
+ askPassword(parent: Component, action: String, error: String) : String
+ showSlip(parent: Component, title: String, lines: List<String>) : void
- printSlip(dialog: JDialog, text: JTextArea, status: JLabel, print: JButton) : void
- reasonOf(error: Throwable) : String
```

«abstract»
BasePanel
ics.gui · extends JPanel

```
- MAX_PASSWORD_TRIES : int = 3
- body : JPanel = new JPanel(new BorderLayout())
# statusLabel : JLabel = Theme.label(" ", Theme.FONT_LABEL, Theme.SUCCESS)
- protectedMessage : String = ""

# BasePanel(title: String, subtitle: String)
+ onShow() : void
# addField(form: JPanel, index: int, label: JLabel, field: JComponent) : void
# askAdminPassword(actionText: String, error: String) : String
# confirm(message: String) : boolean
# getBody() : JPanel
# getProtectedMessage() : String
# onKey(keyStroke: String, action: Runnable) : void
# runProtected(actionText: String, action: BasePanel.ProtectedAction) : BasePanel.Outcome
# setStatus(text: String, ok: boolean) : void
```

«enumeration»
BasePanel.Outcome
ics.gui

```
DONE
CANCELLED
FAILED
```

«interface»
BasePanel.ProtectedAction
ics.gui

```
+ run(adminPassword: String) : void
```

LoginPanel
ics.gui · extends JPanel

```
- MAX_TRIES : int = 3
- auth : AuthService
- onSuccess : LoginPanel.LoginListener
- onTerminate : Runnable
- usernameField : JTextField = Theme.styleField(new JTextField())
- passwordField : JPasswordField = Theme.styleField(new JPasswordField())
- errorLabel : JLabel = Theme.label(" ", Theme.FONT_LABEL, Theme.ERROR)
- loginButton : Widgets.RoundedButton = new Widgets.RoundedButton("Log in", Widgets.RoundedButton.Kind.PRIMARY)
- wrongTries : int

+ LoginPanel(auth: AuthService, onSuccess: LoginPanel.LoginListener, onTerminate: Runnable)
+ addNotify() : void
# announce(message: String) : void
- attemptLogin() : void
- left(c: JComponent) : JComponent
- setError(text: String) : void
```

«interface»
LoginPanel.LoginListener
ics.gui

```
+ loggedIn(user: User) : void
```

ShellPanel
ics.gui · extends JPanel

```
- cards : CardLayout = new CardLayout()
- content : JPanel = new JPanel(cards)
- pages : Map<String, BasePanel> = new LinkedHashMap<>()
- navButtons : List<Widgets.NavButton> = new ArrayList<>()

+ ShellPanel(store: DataStore, user: User, onLogout: Runnable)
+ show(key: String) : void
- buildSidebar(user: User, onLogout: Runnable) : JPanel
- sectionHeading(text: String) : JLabel
```

DashboardPanel
ics.gui · extends BasePanel

```
- shell : ShellPanel
- user : User
- auth : AuthService
- users : UserService
- policies : PolicyService
- claims : ClaimService
- numbers : JLabel[] = new JLabel[4]
- hints : JLabel[] = new JLabel[4]
- replyHint : JLabel = Theme.label(" ", Theme.FONT_LABEL, Theme.AMBER)
- passwordHint : JLabel = Theme.label(" ", Theme.FONT_LABEL, Theme.ERROR)

+ DashboardPanel(shell: ShellPanel, user: User, auth: AuthService, users: UserService, policies: PolicyService, claims: ClaimService)
+ onShow() : void
- shortcut(text: String, screen: String, key: int, kind: Widgets.RoundedButton.Kind) : Widgets.RoundedButton
- show(card: int, number: int, hint: String) : void
```

UserPanel
ics.gui · extends BasePanel

```
- service : UserService
- admin : User
- selected : User
- formTitle : JLabel = Theme.label("New account", Theme.FONT_HEADING, Theme.TEXT_DARK)
- roleBox : JComboBox<User.Role> = new JComboBox<>(User.Role.values())
- usernameField : JTextField = Theme.styleField(new JTextField())
- fullNameField : JTextField = Theme.styleField(new JTextField())
- contactField : JTextField = Theme.styleField(new JTextField())
- addressField : JTextField = Theme.styleField(new JTextField())
- passwordField : JPasswordField = Theme.styleField(new JPasswordField())
- confirmField : JPasswordField = Theme.styleField(new JPasswordField())
- fieldsByName : Map<String, JComponent> = new HashMap<>()
- addButton : Widgets.RoundedButton = new Widgets.RoundedButton("Add user", Widgets.RoundedButton.Kind.PRIMARY)
- updateButton : Widgets.RoundedButton = new Widgets.RoundedButton("Update", Widgets.RoundedButton.Kind.PRIMARY)
- clearButton : Widgets.RoundedButton = new Widgets.RoundedButton("Clear", Widgets.RoundedButton.Kind.SECONDARY)
- blockButton : Widgets.RoundedButton = new Widgets.RoundedButton("Deactivate", Widgets.RoundedButton.Kind.SECONDARY)
- deleteButton : Widgets.RoundedButton = new Widgets.RoundedButton("Delete", Widgets.RoundedButton.Kind.SECONDARY)
- searchField : JTextField = Theme.styleField(new JTextField())
- countLabel : JLabel = Theme.label("", Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- table : Widgets.Table = new Widgets.Table(new String[] {"ID", "Username", "Full name", "Role", "Contact", "Stat…

+ UserPanel(service: UserService, admin: User)
+ onShow() : void
- addUser() : void
- buildForm() : void
- buildTable() : void
- clearForm() : void
- deactivateUser() : void
- deleteUser() : void
- loadSelected() : void
- refreshTable() : void
- setEditing(editing: boolean) : void
- showFieldError(e: InvalidInputException) : void
- unblockUser() : void
- updateUser() : void
- wireActions() : void
```

PolicyPanel
ics.gui · extends BasePanel

```
- service : PolicyService
- admin : User
- selected : Policy
- formTitle : JLabel = Theme.label("Issue a policy", Theme.FONT_HEADING, Theme.TEXT_DARK)
- typeBox : JComboBox<Policy.Type> = new JComboBox<>(Policy.Type.values())
- holderBox : JComboBox<PolicyPanel.HolderItem> = new JComboBox<>()
- startField : JTextField = Theme.styleField(new JTextField())
- endField : JTextField = Theme.styleField(new JTextField())
- deductibleField : JTextField = Theme.styleField(new JTextField())
- limitField : JTextField = Theme.styleField(new JTextField())
- percentField : JTextField = Theme.styleField(new JTextField())
- waitingField : JTextField = Theme.styleField(new JTextField())
- vehicleField : JTextField = Theme.styleField(new JTextField())
- plateField : JTextField = Theme.styleField(new JTextField())
- percentLabel : JLabel = Theme.requiredLabel("Reimbursement (%)")
- waitingLabel : JLabel = Theme.requiredLabel("Waiting period (days)")
- vehicleLabel : JLabel = Theme.requiredLabel("Vehicle value")
- plateLabel : JLabel = Theme.requiredLabel("Plate number")
- premiumLabel : JLabel = Theme.label(" ", Theme.FONT_BUTTON, Theme.NAVY)
- fieldsByName : Map<String, JComponent> = new HashMap<>()
- issueButton : Widgets.RoundedButton = new Widgets.RoundedButton("Issue", Widgets.RoundedButton.Kind.PRIMARY)
- updateButton : Widgets.RoundedButton = new Widgets.RoundedButton("Update", Widgets.RoundedButton.Kind.PRIMARY)
- clearButton : Widgets.RoundedButton = new Widgets.RoundedButton("Clear", Widgets.RoundedButton.Kind.SECONDARY)
- cancelButton : Widgets.RoundedButton = new Widgets.RoundedButton("Cancel policy", Widgets.RoundedButton.Kind.SECONDARY)
- deleteButton : Widgets.RoundedButton = new Widgets.RoundedButton("Delete", Widgets.RoundedButton.Kind.SECONDARY)
- searchField : JTextField = Theme.styleField(new JTextField())
- countLabel : JLabel = Theme.label("", Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- table : Widgets.Table = new Widgets.Table(new String[] {"ID", "Holder", "Type", "Status", "Ends", "Limit left"}…

+ PolicyPanel(service: PolicyService, admin: User)
+ onShow() : void
- buildForm() : void
- buildTable() : void
- cancelPolicy() : void
- clearForm() : void
- deletePolicy() : void
- issuePolicy() : void
- loadSelected() : void
- plain(value: double) : String
- refreshTable() : void
- setEditing(editing: boolean) : void
- showFieldError(e: InvalidInputException) : void
- showTypeFields() : void
- updatePolicy() : void
- updatePremium() : void
- wireActions() : void
```

PolicyPanel.HolderItem
ics.gui

```
~ user : User

~ HolderItem(user: User)
+ toString() : String
```

ReviewPanel
ics.gui · extends BasePanel

```
- ADMIN_REASONS : Claim.Reason[] = { Claim.Reason.EXCLUDED_CAUSE, Claim.Reason.INSUFFICIENT_DOCUMENTS, Claim.Reason.SUSPEC…
- service : ClaimService
- admin : User
- shown : List<Claim> = new ArrayList<>()
- selectedId : String
- filterBox : JComboBox<ClaimService.Filter> = new JComboBox<>(ClaimService.Filter.values())
- searchField : JTextField = Theme.styleField(new JTextField())
- countLabel : JLabel = Theme.label("", Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- table : Widgets.Table = new Widgets.Table(new String[] {"ID", "Holder", "Claimed", "Status", "Filed"}, new int[…
- detailCards : CardLayout = new CardLayout()
- detailHolder : JPanel = new JPanel(detailCards)
- titleLabel : JLabel = Theme.label(" ", Theme.FONT_HEADING, Theme.TEXT_DARK)
- chip : JLabel = Widgets.chip()
- factsLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- descriptionArea : JTextArea = new JTextArea(3, 20)
- replyHeading : JLabel = Theme.fieldLabel("Reply from the user")
- replyLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.NAVY)
- payoutHeading : JLabel = Theme.fieldLabel("How the payout would be worked out")
- payoutLabel : JLabel = Theme.label(" ", Theme.FONT_CAPTION, Theme.TEXT_DARK)
- decisionPanel : JPanel = new JPanel()
- reasonBox : JComboBox<Claim.Reason> = new JComboBox<>(ADMIN_REASONS)
- noteArea : JTextArea = new JTextArea(3, 20)
- savedNoteHeading : JLabel = Theme.fieldLabel("Note to the user")
- savedNoteLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- buttonRow : JPanel = new JPanel(new GridLayout(1, 0, 8, 0))
- approveButton : Widgets.RoundedButton = button("Approve", KeyEvent.VK_A, true, "Approve and release the payout shown above (Alt…
- denyButton : Widgets.RoundedButton = button("Deny", KeyEvent.VK_D, false, "Deny with the reason chosen above (Alt+D)")
- infoButton : Widgets.RoundedButton = button("Ask for info", KeyEvent.VK_I, false, "Send your note to the user and wait for t…
- resumeButton : Widgets.RoundedButton = button("Resume review", KeyEvent.VK_R, true, "Continue without waiting for the user's r…
- slipButton : Widgets.RoundedButton = button("View slip", KeyEvent.VK_V, false, "Show and print the payout slip (Alt+V)")
- closeButton : Widgets.RoundedButton = button("Close claim", KeyEvent.VK_C, false, "Finish this claim (Alt+C)")

+ ReviewPanel(service: ClaimService, admin: User)
+ onShow() : void
- addButtons(buttons: JButton...) : void
- afterChange() : void
- approve() : void
- askForInfo() : void
- buildDetail() : void
- buildList() : void
- button(text: String, key: int, primary: boolean, tip: String) : Widgets.RoundedButton
- deny() : void
- left(c: JComponent) : JComponent
- loadDetail() : void
- openSlip(id: String) : void
- refreshList() : void
# showSlip(claimId: String, lines: List<String>) : void
- step(doneMessage: String, action: ReviewPanel.Step) : void
- wireActions() : void
```

«interface»
ReviewPanel.Step
ics.gui

```
+ run() : void
```

ApplyPanel
ics.gui · extends BasePanel

```
- MAX_DESCRIPTION : int = 300
- service : ClaimService
- applicant : User
- pendingPolicyId : String
- policyBox : JComboBox<ApplyPanel.PolicyItem> = new JComboBox<>()
- incidentField : JTextField = Theme.styleField(new JTextField())
- amountField : JTextField = Theme.styleField(new JTextField())
- amountLabel : JLabel = Theme.requiredLabel("Claimed amount")
- descriptionArea : JTextArea = new JTextArea(5, 20)
- counterLabel : JLabel = Theme.label("0 / " + MAX_DESCRIPTION, Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- submitButton : Widgets.RoundedButton = new Widgets.RoundedButton("Submit claim", Widgets.RoundedButton.Kind.PRIMARY)
- clearButton : Widgets.RoundedButton = new Widgets.RoundedButton("Clear", Widgets.RoundedButton.Kind.SECONDARY)
- fieldsByName : Map<String, JComponent> = new HashMap<>()
- termsLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- estimateLabel : JLabel = Theme.label(" ", Theme.FONT_BUTTON, Theme.NAVY)
- warningLabel : JLabel = Theme.label(" ", Theme.FONT_LABEL, Theme.AMBER)

+ ApplyPanel(service: ClaimService, applicant: User)
+ onShow() : void
+ preselect(policyId: String) : void
- buildUi() : void
- clearForm() : void
# confirm(message: String) : boolean
- left(c: JComponent) : JComponent
- refreshPreview() : void
- reloadPolicies() : void
- showCheck(fill: Color, text: Color, message: String) : void
- submit() : void
- updateCounter() : void
- wireActions() : void
```

ApplyPanel.PolicyItem
ics.gui

```
~ policy : Policy
~ label : String

~ PolicyItem(policy: Policy, today: LocalDate)
+ toString() : String
```

MyPoliciesPanel
ics.gui · extends BasePanel

```
- service : ClaimService
- applicant : User
- onApply : MyPoliciesPanel.ApplyAction
- shown : List<Policy> = new ArrayList<>()
- selectedId : String
- table : Widgets.Table = new Widgets.Table(new String[] {"ID", "Type", "Status", "Ends", "Remaining"}, new int[]…
- countLabel : JLabel = Theme.label("", Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- detailCards : CardLayout = new CardLayout()
- detailHolder : JPanel = new JPanel(detailCards)
- titleLabel : JLabel = Theme.label(" ", Theme.FONT_HEADING, Theme.TEXT_DARK)
- chip : JLabel = Widgets.chip()
- termsLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- usageBar : Widgets.UsageBar = new Widgets.UsageBar()
- usageLabel : JLabel = Theme.label(" ", Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- claimsLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- applyButton : Widgets.RoundedButton = new Widgets.RoundedButton("Apply for a claim", Widgets.RoundedButton.Kind.PRIMARY)

+ MyPoliciesPanel(service: ClaimService, applicant: User, onApply: MyPoliciesPanel.ApplyAction)
+ onShow() : void
- buildUi() : void
- claimsSummary(p: Policy) : String
- left(c: JComponent) : JComponent
- showDetail(p: Policy) : void
```

«interface»
MyPoliciesPanel.ApplyAction
ics.gui

```
+ applyOn(policyId: String) : void
```

MyClaimsPanel
ics.gui · extends BasePanel

```
- service : ClaimService
- applicant : User
- shown : List<Claim> = new ArrayList<>()
- selectedId : String
- table : Widgets.Table = new Widgets.Table(new String[] {"ID", "Claimed", "Status", "Filed"}, new int[] {82, 115…
- countLabel : JLabel = Theme.label("", Theme.FONT_CAPTION, Theme.TEXT_MUTED)
- detailCards : CardLayout = new CardLayout()
- detailHolder : JPanel = new JPanel(detailCards)
- titleLabel : JLabel = Theme.label(" ", Theme.FONT_HEADING, Theme.TEXT_DARK)
- chip : JLabel = Widgets.chip()
- progress : Widgets.ClaimProgress = new Widgets.ClaimProgress()
- summaryLabel : JLabel = Theme.label(" ", Theme.FONT_LABEL, Theme.TEXT_DARK)
- factsLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- adminNoteHeading : JLabel = Theme.fieldLabel("Note from the Admin")
- adminNoteLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- replyHeading : JLabel = Theme.fieldLabel("Your reply")
- sentReplyLabel : JLabel = Theme.label(" ", Theme.FONT_BODY, Theme.TEXT_DARK)
- replyArea : JTextArea = new JTextArea(3, 20)
- replyScroll : JScrollPane = new JScrollPane(replyArea)
- buttonRow : JPanel = new JPanel(new GridLayout(1, 0, 8, 0))
- replyButton : Widgets.RoundedButton = new Widgets.RoundedButton("Send reply", Widgets.RoundedButton.Kind.PRIMARY)
- slipButton : Widgets.RoundedButton = new Widgets.RoundedButton("View payout slip", Widgets.RoundedButton.Kind.PRIMARY)

+ MyClaimsPanel(service: ClaimService, applicant: User)
+ onShow() : void
- buildUi() : void
- left(c: JComponent) : JComponent
- loadDetail() : void
- refreshList() : void
- sendReply() : void
# showSlip(claimId: String, lines: List<String>) : void
- summary(c: Claim) : String
- userLabel(status: Claim.Status) : String
```

ChangePasswordPanel
ics.gui · extends BasePanel

```
- auth : AuthService
- user : User
- currentField : JPasswordField = Theme.styleField(new JPasswordField())
- newField : JPasswordField = Theme.styleField(new JPasswordField())
- confirmField : JPasswordField = Theme.styleField(new JPasswordField())
- changeButton : Widgets.RoundedButton = new Widgets.RoundedButton("Change password", Widgets.RoundedButton.Kind.PRIMARY)
- fieldsByName : Map<String, JPasswordField> = new HashMap<>()

+ ChangePasswordPanel(auth: AuthService, user: User)
+ onShow() : void
- change() : void
```